21 April 2026 09:40

Exceptions:

An exception is an error that happens during program execution.

Instead of stopping the program suddenly , python gives an exception which we can handle by using
- Try
- Except

Example
```
a=10
b=0
print(a/b)
print("end of the program")
```

```
Traceback (most recent call last):
  File "D:\Myworkspace\day6.py", line 3, in <module>
    print(a/b)
        ~^~
ZeroDivisionError: division by zero
```

Why exceptions are important?
Used to:
- Prevent application crashes
- Handle invalid user inputs
- Log errors
- Retry failed operations
- Useful in devops automation scripts

Sytax:-
Try:
Risk code()
Except:
Handle code()

Flow of try, except--:
Try block is always executes.
Except: block will execute only when the try block throws an error .
If any error --> except will executes
If no errors --> skip except ;
For the best practice always specify particular error

Value error:- when irrelevant of value is assigning to the variable then we are getting this type of error
```
try:
    number=int(input("enter the mobile number"))
    print(number)
except ValueError:
    print("invalid number")
```

```
Multiple exception:

try:
    x=int(input("enter numbe:"))
    div=10/x
    print("the result is ",div)
except ValueError:
    print(" enter only the integer values ")
except ZeroDivisionError:
    print("cannot devide by zero.....")
```

```
Using exception object:--


try:
    x=int(input("enter numbe:"))
    div=10/x
    print("the result is ",div)
except Exception as e:
    print€

# full syntax for exceptions:


try:
    code()
except error1:
    code:
except error2:
    code:
else:
    code
finally:
    code
```

else block will runs no exceptions occur

```
try:
    x=int(input("enter numbe:"))
    div=10/x

except Exception as e:
    print(e)
else:
     print("the result is ",div)
```

Finally:- this block will executes always ,
Used to:-

- Closing the file
- Database connections closes
- Clean up task

Common built-in exceptions:-

Exception	Meaning
zeroDivisionError	Divide by zero (10/0)
ValueError	Invalid values (int("10))
TypeError	Wrong data type ("10" + 20)
NameError	Variable not defined
IndexError	Invalid list in the index a=[1,2] print(a[5])
KeyError	Missing the dictionary key
FileNotFoundError	File missing
ImportErro	Module missing

Raising exceptions manually:-
Use raise-

Age= -1
If age<0:
Raise valueError("age cannot be negative)

OOP OBJECT ORIENTED PROGRAMING:-\

Way of designing a software or applications using objects and that consist of
Data(attribute /properties)
Behavior (methods/functions)
Example:-
A server can have :-

- Name
- Ip address
- OS type
- Methods like start(),stop(),restart()

Why OOPs is important:-

- Reusability (write code once and reuse many times)

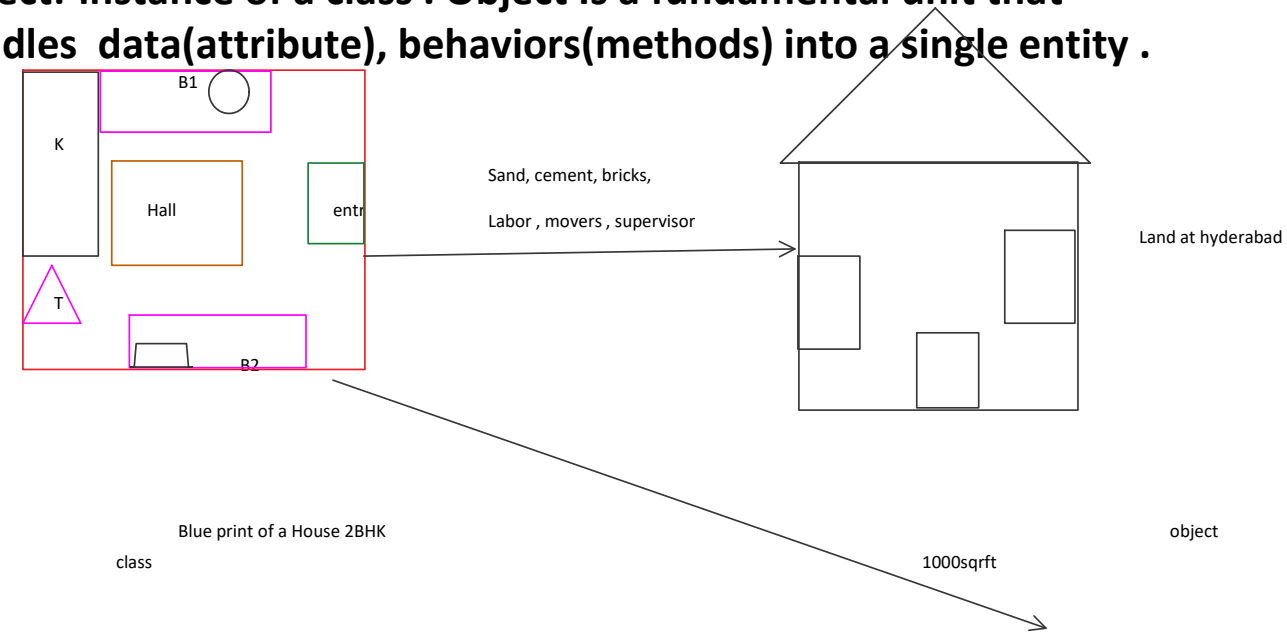
- **Maintainability (change can be made in one class instead of many times)**
- **Scalability (easy to expand systems)**
- **Security (sensitive data can be protected)**
- **Readability (organize the code and understandable)**

Where OOPs used in real time:-

Banking software
e-commerce application
Cloud platforms
Devops automations
Networking tools
Monitoring systems

Class and object:-

Class is a blueprint or template , for creating objects , which bundled with data(attributes), functionalities (methods) together
Object:-instance of a class . Object Is a fundamental unit that bundles data(attribute), behaviors(methods) into a single entity .



Syntax:-
Class class_name:
    Block of code
    Attributes
    Constructor
    A method(behaviour)

Object_name=classname()

Objects having 3 core properties --:
Identity :-a unique integer that represents object memory location
Type(): defines what the object can do and what values it can hold (like, int,str..)
Value:-the actual data stored with in the object

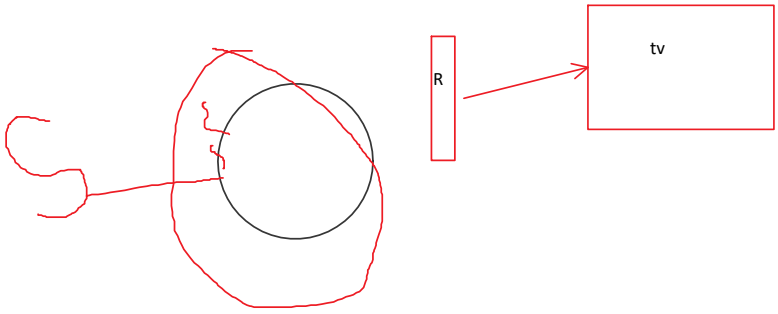
We can create multiple objects with single class

3.types of variables :-

1. **Instance variables (object level variable)**

2. **Static variables (class level variables)**
3. **Local variable (method level)**

```
class Student:
    def __init__(self):
        self.name="murlai"
        self.rollno=101
        self.mark=90
    def details(self):
        print("hello i am ",self.name)
        print("my roll number ",self.rollno)
        print("marks  are ",self.mark)
s=Student()
s.details()
# print(s.name)
# print(s.rollno)
```



Constructor:--

In python , constructor is a special method used to initialize a new object's attributes .it is automatically executes when an instance of a class is created
Most commonly used constructor is __init__(self): within the class
Constructor can return the non-value ..

When creation of every object , the constructor is called each time automatically

Purpose constructor is to allocate memory and storage

```
class Student:
    def __init__(self):
        print("constructor execution ....")
        self.name="murlai"
        self.rollno=101
        self.mark=90
    def details(self):
        print("hello i am ",self.name)
        print("my roll number ",self.rollno)
        print("marks  are ",self.mark)
s=Student()
s2=Student()
s3=Student()
print(id(s))
print(id(s2))
```

```
Student() :
 1. We are creating an object . PVM will create object and allocate memory
 2. PVM will execute __inti__() , to declare and initialize variables of that object (instance variable)

s=Student() : for newly created and initialized objects , we are assigning  s referee variable .
By using we can able to access variables, methods on that object
```

Self- it can hold reference of current object

```
class Test:
    def __init__(self):
        print("address of object refered by self:,",id(self))
t=Test()
print("Address of object referenced b T",id(t))
```

```
address of object refered by self:, 30379088
Address of object referenced b T 30379088
```

Within the python class , to access current object , we can use self.

First argument to the constructor is always self
First argument to the instance method is always self
We are not required to provide value for self,PVM (python virtual machine) will provide value

We can use sefl always with in the class only
Inside constructor , we can use self to decare object realted variables(instance variables)

Self is not a key word, we can give any name like, first , last …

4 pillars of Oops :-

1. **Encapsulation**
2. **Inheritance**
3. **Polymorphism**
4. **Abstraction**

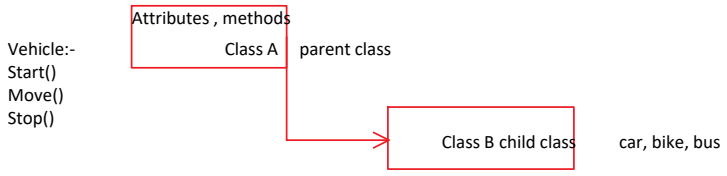
Encapsulation:- wrapping data(variable,) and methods(functions) together inside a class , and controlling direct access to data (protects) internal data

Class account:
Def __init__(self,balance)
Self.__balance=balance
__balance--> it makes private(name-mangled)
Used in:- API key
Token
Password
Ssh credentials
Salary info
Internal configu
Benefits:-
Security
Data protection
Controlled access
Clean design

Inheritance :- child class gets properties from parent class . Write one common code and we can reuse this many times …

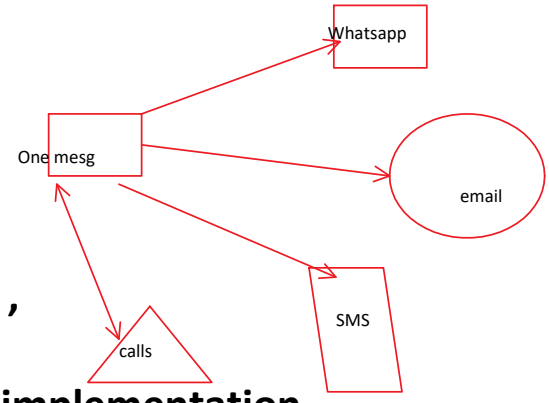
Vehicle:- Attributes , methods Class A parent class

Write one common code and we can reuse this many times ...



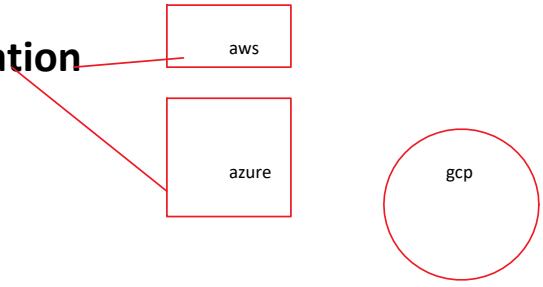
Whenever inheritance happened , the object creation always done for child class
why we need to use inheritance ..>
To avoid repeat of methods in every class
Write once , use many times

Polymorphism:=-=



Same method name ,
Different behavior's
One interface many implementation

Deploy application



Abstraction:-
Show only essential operations and hide implementation details:
Creating of virtual machines
Delete vm
Scale
Backup vm '

```
#encapsulation:-
# class Employee:
#     def __init__(self,name,salary):
#         self.name=name
#         self.__salry=salary #private
#     def show_salary(self):
#         print("salary :",self.__salry)
#     def increase_salary(self,amount):
#         self.__salry+=amount
#         print("salary updated ")
# e1=Employee("ravi ",500000)
# e1.show_salary()
# e1.increase_salary(5000)
# e1.show_salary()
#polymorphism
# class UPI:
#     def pay(self):
#         print("paid using UPI")
# class Card:
#     def pay(self):
```

```
#         print("paid using CARD")
# class Netbanking:
#     def pay(self):
#         print("paid using Netbanking")
# method=[UPI(),Card(),Netbanking()]
# for m in method:
#     m.pay()
class AWS:
    def __init__(self):
        pass
    def deploy(self):
        print("deployed to AWS")
class Azure:
    def deploy(self):
        print("deployed to Azure")
class GCP:
    def deploy(self):
        print("deployed to GCP")
ob=AWS()
ob.deploy()
# provider=[AWS(),Azure(),GCP()]
# AWS.deploy()
# # aws.deploy()
# # a=Azure()
# # a.deploy()
# gcp=GCP()
# # gcp.deploy()
```